

Q1

1.

	Good Prediction	Bad Prediction
MSE	0.375	0.188
BSE	0.197	0.568

For good predictions:

- Both MSE and BCE are small.

For bad predictions:

- MSE increases moderately.
- BCE increases much more sharply.

BCE grows much more when predictions are wrong compared to MSE.

2.

Binary Cross Entropy (BCE) penalizes wrong confident predictions much more strongly.

Example:

If $y = 1$ and prediction = 0.01

- $MSE = (1 - 0.01)^2 \approx 0.98$
- $BCE = -\log(0.01) \approx 4.6$ (very large)

So BCE heavily punishes confident wrong predictions

3.

Because:

1. It is derived from maximum likelihood estimation.
2. It matches Bernoulli distribution.
3. It produces better gradients when used with sigmoid.

MSE treats classification like regression — not ideal.

Q1

4.

Sigmoid

$$g = \sigma(z) = \frac{1}{1 + e^{-z}}$$

BCE Loss

$$L = -[y \log(\hat{y}) + (1-y) \log(1-\hat{y})]$$

Compute

$$\frac{\partial L}{\partial z}$$

Chain Rule

$$\frac{\partial L}{\partial z} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z}$$

$$\frac{\partial L}{\partial z} = \hat{y} - y$$

Why is this powerful

Because Sigmoid derivative cancels out,

If we used MSE + Sigmoid

$$(\hat{y} - y) \hat{y} (1 - \hat{y})$$

This makes gradients very small when sigmoid saturates

BCE + Sigmoid avoids vanishing gradients in the output layer

This improves Numerical stability

5.

For regression → MSE is appropriate.

Because:

- Output is continuous
- Errors measured in Euclidean distance
- Gaussian noise assumption

Q2

1.Adam ,Because it adapts learning rate per parameter.

2.Adam is usually smoother due to adaptive moment estimation.

3.Adam computes:

- First moment (mean of gradients)
- Second moment (variance of gradients)

It rescales gradients:

So:

- Large gradients are scaled down
- Small gradients are scaled up

This improves convergence.

4.Yes.

Sometimes:

- Adam finds sharp minima
- SGD finds flatter minima

Flatter minima → better generalization.

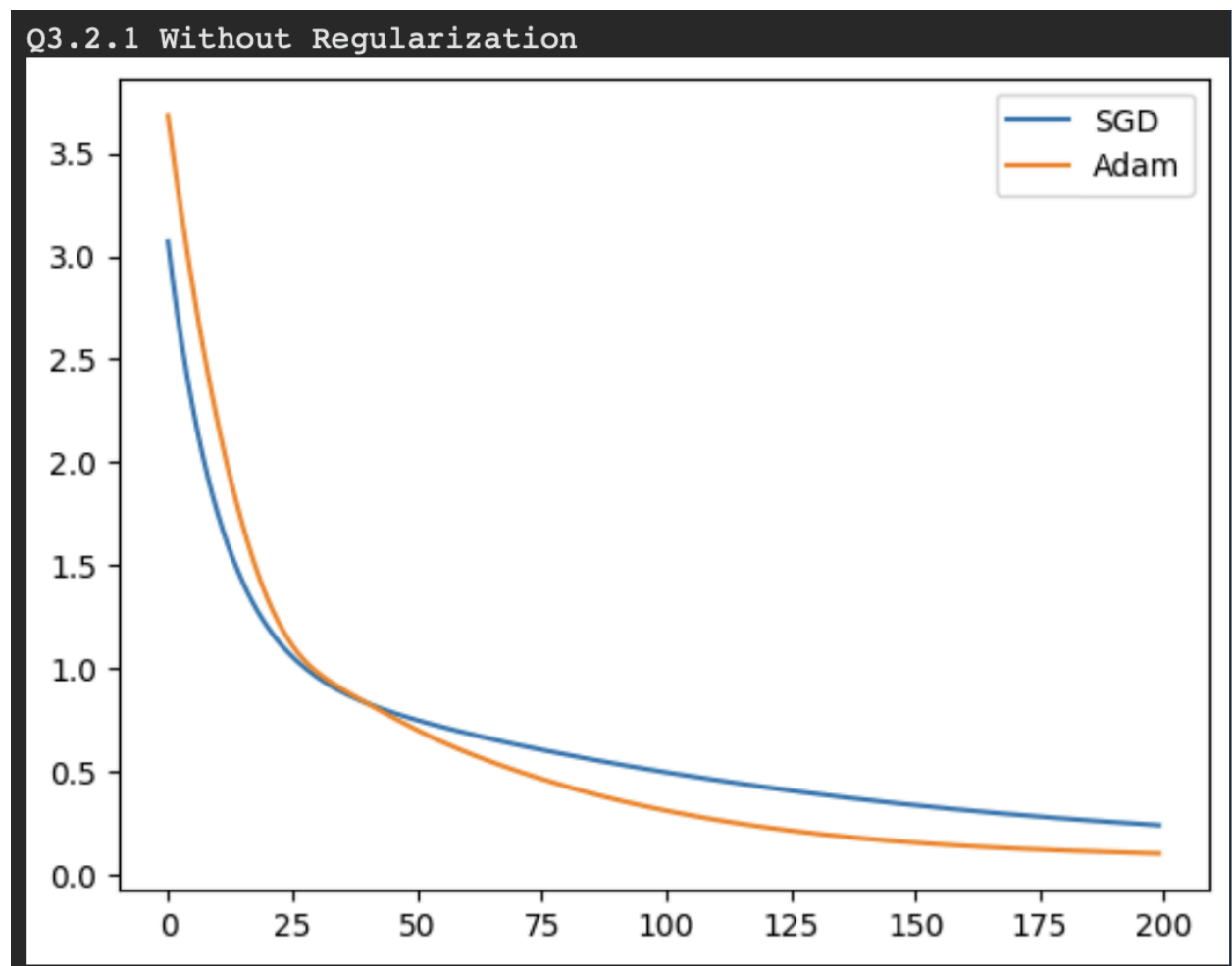
So faster is not always better.

5.

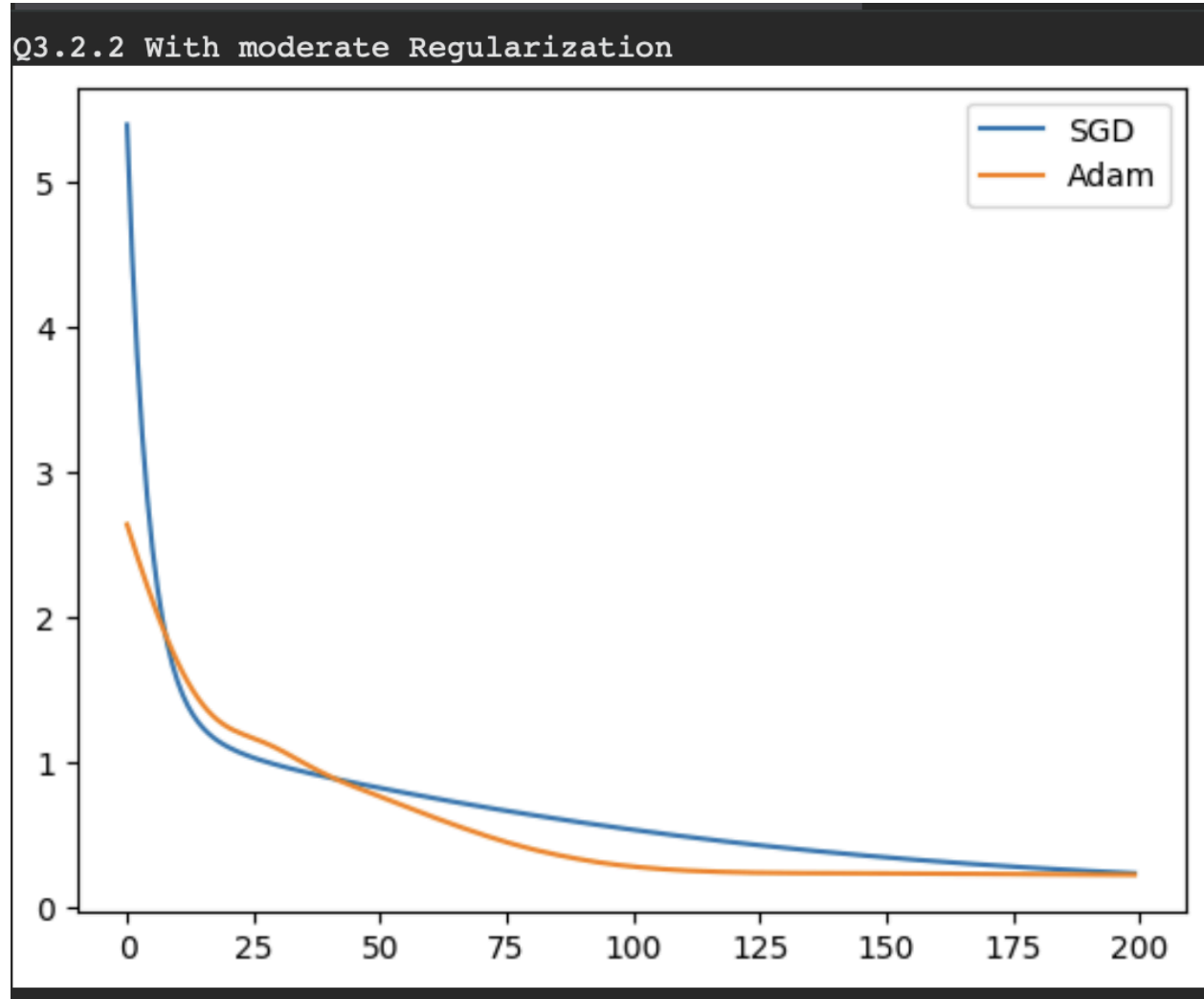
Loss Function	Optimizer
Defines what to minimize	Defines how to minimize
Measures Error	Updates Parameteres
Shapes Objective Geometry	Defines Trajectory in Parameter in space

Q3

2.1

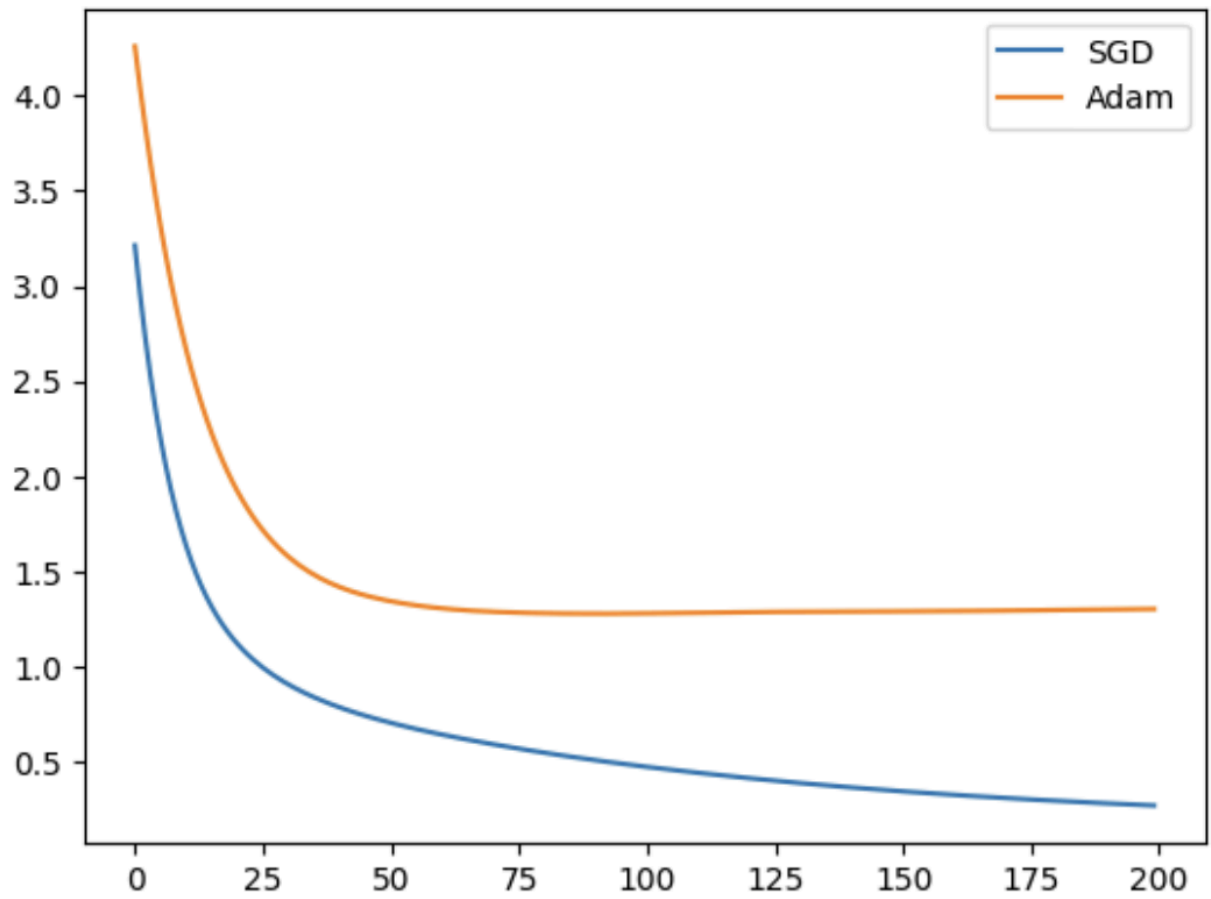


2.2



2.3

Q3.2.2 With Strong Regularization



1.

No Regularization	Lowest Training Loss
Moderate Regularization	Slightly Higher loss
Strong Regularization	Higest Loss

2,3,4 and 5

Q3

2. Increases training loss

Because adding penalty

$$L_{total} = L(\theta) + \lambda \|\theta\|_2^2$$

So model cannot freely minimize only empirical loss.

Q3

3

$$L_{total}(\theta) = L(\theta) + \lambda \|\theta\|_2^2$$

4. Original gradient

$$\nabla_{\theta} L(\theta)$$

with L_2

$$\nabla_{\theta} L_{total} = \nabla_{\theta} L(\theta) + 2\lambda\theta$$

update rule

$$\theta_{t+1} = \theta_t - \eta(\nabla_{\theta} L + 2\lambda\theta)$$

This is called weight decay

5. L_1 Penalty

$$\lambda \|\theta\|_1$$

Gradient is constant magnitude ($\pm \lambda$)

This, pushes small weight exactly to zero.

L_2 shrinks weights gradually but rarely to zero

6.

Because:

- Model capacity effectively reduced
- Weights forced too small
- Model cannot capture true pattern

High bias.

7. $\text{Error} = \text{Bias}^2 + \text{Variance} + \text{Noise}$

Regularization:

- Increases bias
- Decreases variance

Moderate regularization → best balance

Too much → high bias (underfitting)